

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>Н2</u>	<u>Программная инженерия и интеллектуальные системы</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Проектирование архитектуры программных средств</u>	
	наименование дисциплины	

УЧЕБНО–ПРАКТИЧЕСКАЯ РАБОТА

5

номер задания (при наличии)

ДИАГРАММА КЛАССОВ

при наличии указать тему учебно–практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О729Б

Барышников О.А.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Зими́на Д.В.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

г. Санкт–Петербург

20 26 г.

СОДЕРЖАНИЕ

1 Диаграмма состояний.....	3
2 Описание диаграммы состояний	4

1 Диаграмма классов

На рисунке 1 изображена диаграмма классов.

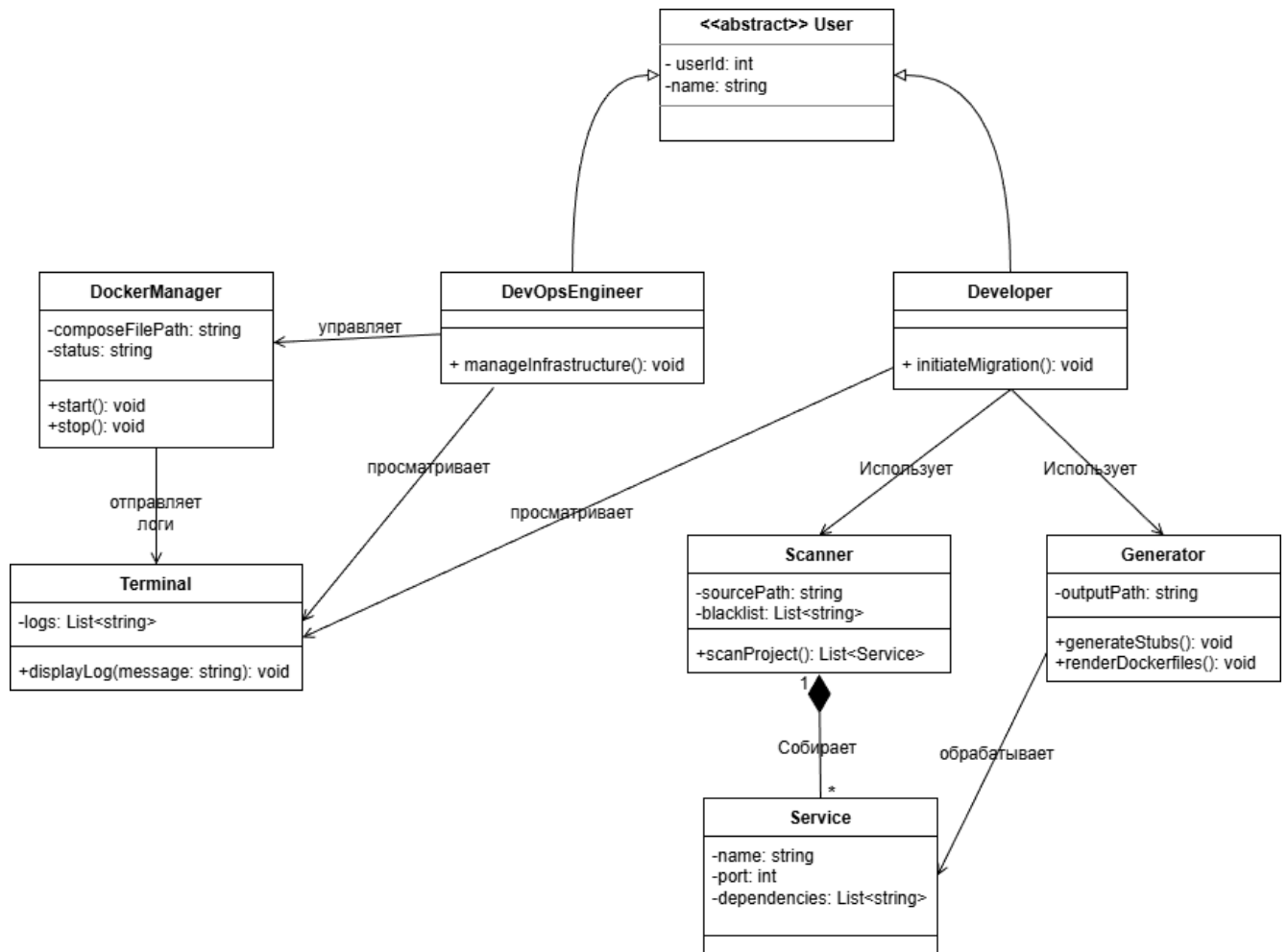


Рисунок 1 – Диаграмма классов

2 Описание диаграммы классов

Разработанная диаграмма классов представляет собой структурную модель программного модуля визуального обнаружения объектов, выполненную в нотации UML. Диаграмма отражает статическую структуру системы, включая основные сущности, их атрибуты, методы и взаимосвязи.

Центральным элементом системы является класс `Scanner`, который содержит пути к исходному коду (`sourcePath`) и список исключений (`blacklist`), а также основной метод эвристического обхода файловой системы `scanProject()`. Класс `Scanner` в процессе работы обнаруживает и создает множество объектов класса `Service`. Каждый объект `Service` представляет собой отдельный независимый микросервис и инкапсулирует в себе такие атрибуты, как имя сервиса (`name`), назначаемый сетевой порт (`port`) и список зависимостей (`dependencies`). Отношение между `Scanner` и `Service` представляет собой композицию «один ко многим» (сканер содержит и формирует набор сервисов).

Для физического разделения кода и создания артефактов используется класс `Generator`. Он содержит пути сохранения конфигураций (`outputPath`) и взаимодействует со списком объектов `Service`, реализуя методы очистки моделей базы данных (`generateStubs()`) и создания инфраструктурных файлов (`renderDockerfiles()`). Класс `Developer` управляет классами `Scanner` и `Generator`, иницилируя процесс миграции.

Подсистема оркестрации представлена классом `DockerManager`, который содержит путь к единому файлу сборки (`composeFilePath`) и текущий статус системы (`status`). Этот класс предоставляет методы управления жизненным циклом контейнеров, такие как `start()` и `stop()`. Класс `DevOpsEngineer` взаимодействует с `DockerManager` для развертывания инфраструктуры.

Для обратной связи с пользователем предусмотрен класс `Terminal`, имеющий атрибут буфера логов (`logs`) и метод эвристической раскраски вывода `displayLog()`. Класс `DockerManager` передает поток вывода в `Terminal` (отношение зависимости), а пользователи могут просматривать эти данные в графическом интерфейсе.

Таким образом, разработанная диаграмма классов охватывает ключевые сущности предметной области, инкапсулирует бизнес-логику внутри соответствующих модулей и позволяет наглядно описать статическую структуру приложения AMM-Docker.